## Summary of the Multi-Agent Reflective Research System

This agent is an **AI research assistant composed of multiple specialized agents** that work together to answer a user's question with quality control and human supervision.

### Workflow

1. **Planner Agent**
   - Breaks the user's question into a structured research plan.
2. **Research Agent**
   - Converts the plan into a short search query.
   - Uses **web search (Tavily)** to gather information.
   - Generates a research summary using **Gemini**.
3. **Critic Agent**
   - Reviews the research.
   - Identifies missing information, errors, or improvements.
4. **Human-in-the-Loop**
   - The system pauses and asks a human to approve or reject the research.
   - If rejected → the research step runs again.
5. **Writer Agent**
   - Uses the research and critique to generate the **final refined answer**.

### Key Features

- **Multi-agent architecture**
- **Self-reflection (critic agent)**
- **Human approval checkpoint**
- **Web search integration**
- **LangGraph workflow with loops**

### In Simple Terms

It works like a **team of researchers**:

- One **plans the research**
- One **collects information**
- One **reviews the work**
- A **human approves it**
- One **writes the final report**.

```
# Set the TAVILY_API_KEY for web search functionality.
```

```
# Install necessary Python packages for LangChain, LangGraph, Google GenAI, and Tavily.
!pip -q install langchain langgraph langchain-groq tavily-python
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 137.5/137.5 kB 6.0 MB/s eta 0:00:00
```

```
# Load the GEMINI_API_KEY securely from Colab's user data.
from google.colab import userdata
import os
#GEMINI_API_KEY = userdata.get("GEMINI_API_KEY")
os.environ['GROQ_API_KEY'] = userdata.get("GROQ_API_KEY")
```

```
# Import required libraries and define TypedDict for state management in LangGraph.
from typing import TypedDict
from langgraph.graph import StateGraph, END
#from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_groq import ChatGroq

from tavily import TavilyClient
```

```
# Initialize the ChatGoogleGenerativeAI model with a specific model, API key, and temperature.
# llm = ChatGoogleGenerativeAI(
#     model="gemini-2.5-flash",
#     google_api_key=GEMINI_API_KEY,
#     temperature=0.2
# )

llm = ChatGroq(
    model="openai/gpt-oss-20b", # Specify the model to use from Groq
    temperature=0 # Set temperature to 0 for deterministic outputs
)
```

```
# Initialize the TavilyClient for web searches and define a `web_search` function.
tavily = TavilyClient(api_key=TAVILY_API_KEY)

def web_search(query):

    # Tavily limit protection
    query = query[:350]

    results = tavily.search(
        query=query,
        max_results=5
    )

    return str(results)
```

```
# Define the AgentState using TypedDict to represent the state of the research workflow.
class AgentState(TypedDict):

    question: str
    plan: str
    research: str
    critique: str
    approved: bool
    final_answer: str
```

```
# Define the `planner_agent` function, responsible for breaking down a question into a research plan.
def planner_agent(state):

    question = state["question"]

    prompt = f"""
You are a research planner.

Break the following question into a research plan.

Question:
{question}

Return a clear step-by-step plan.
"""

    response = llm.invoke(prompt)

    return {"plan": response.content}
```

```
# Define the `researcher_agent` function, which generates a search query, performs a web search, and summarizes the results.
def researcher_agent(state):

    plan = state["plan"]

    # Step 1: Generate short search query
    query_prompt = f"""
Convert the research plan below into a short web search query.

Plan:
{plan}

Return ONE search query under 15 words.
"""

    query = llm.invoke(query_prompt).content.strip()
    query = query[:350]
```

```
    # Step 2: Web search
    search_results = web_search(query)

    # Step 3: Generate research summary
    research_prompt = f"""
You are a research assistant.

Research Plan:
{plan}

Search Results:
{search_results}

Write a detailed research summary.
"""

    research = llm.invoke(research_prompt)

    return {"research": research.content}
```

```
# Define the `critic_agent` function, responsible for evaluating the research and providing critique.
def critic_agent(state):

    research = state["research"]

    prompt = f"""
You are a research critic.

Evaluate the following research.

Research:
{research}

Identify:
- missing information
- logical errors
- possible improvements

Provide critique.
"""

    critique = llm.invoke(prompt)

    return {"critique": critique.content}
```

```
# Define the `human_review` function, which allows a human to review the research and critique, deciding whether to approve.
def human_review(state):

    print("\n==============================")
    print("HUMAN REVIEW REQUIRED")
    print("==============================")

    print("\nResearch Summary:\n")
    print(state["research"])

    print("\nCritique:\n")
    print(state["critique"])

    decision = input("\nApprove research? (yes/no): ")

    if decision.lower() == "yes":
        return {"approved": True}
    else:
        return {"approved": False}
```

```
# Define the `writer_agent` function, which uses the research and critique to produce a final, improved answer.
def writer_agent(state):

    research = state["research"]
    critique = state["critique"]

    prompt = f"""
You are an expert technical writer.
```

```
Research:
{research}

Critique:
{critique}

Improve the research and produce a final clear answer.
"""

    answer = llm.invoke(prompt)

    return {"final_answer": answer.content}
```

```
# Define the `approval_router` function, which conditionally routes the workflow based on human approval.
def approval_router(state):

    if state["approved"]:
        return "writer"
    else:
        return "researcher"
```

```
# Build and compile the LangGraph workflow, defining the nodes, entry point, and edges between agents.
workflow = StateGraph(AgentState)
 b
workflow.add_node("planner", planner_agent)
workflow.add_node("researcher", researcher_agent)
workflow.add_node("critic", critic_agent)
workflow.add_node("human_review", human_review)
workflow.add_node("writer", writer_agent)

workflow.set_entry_point("planner")

workflow.add_edge("planner", "researcher")
workflow.add_edge("researcher", "critic")
workflow.add_edge("critic", "human_review")

workflow.add_conditional_edges(
    "human_review",
    approval_router,
    {
        "writer": "writer",
        "researcher": "researcher"
    }
)

workflow.add_edge("writer", END)

app = workflow.compile()
```

```
# Define the initial question and invoke the compiled LangGraph workflow to get the final answer.
question = "Impact of hydrogen fuel on future aviation"

result = app.invoke({
    "question": question,
    "plan": "",
    "research": "",
    "critique": "",
    "approved": False,
    "final_answer": ""
})

print("\n==============================")
print("FINAL ANSWER")
print("==============================\n")

print(result["final_answer"])
```

```
| **Long-Haul Feasibility** | The plan focuses on regional/medium haul but does not quantify long-haul range penalties. | Exter

---

### 2. Logical / Methodological Issues

| Issue | Explanation | Risk |
|-------|-------------|------|
| **Zero $CO_2$ Claim** | Claims “zero $CO_2$ during flight” ignores upstream emissions. | Over-optimistic LCA; may mislead policy ma
| **TRL Assumptions** | Fuel cells and tanks at TRL 6–7, combustion engines at TRL 4–5. | Underestimates technical and certifi
| **Infrastructure Cost Estimate** | 10–15 % airport cost increase likely underestimates capital outlay. | Misjudges investment
| **Cost Parity Threshold** | Parity by 2035 if hydrogen < $1.50 $kg^{-1}$ and carbon price > $50 $tCO_2^{-1}$. | Highly sensitive to mar
| **Scenario Development** | Only 2035/2050 scenarios; no adoption-rate spectrum. | Limits policy relevance; cannot capture ea
| **Monte-Carlo Sensitivity** | Variables not specified; risk of missing key drivers. | Incomplete uncertainty quantification.
| **Stakeholder Engagement Scope** | Excludes passengers, airport operators, local communities. | Incomplete view of acceptance
| **Regulatory Review Depth** | No safety standards, certification timelines, or harmonization analysis. | Recommendations may
| **LCA Scope** | System boundaries not fully defined (well-to-wheels vs. cradle-to-grave). | Incomplete environmental assessme

---

### 3. Concrete Improvements & Implementation Plan

| Enhancement | Implementation Steps | Deliverable |
|-------------|----------------------|-------------|
| **Production-Mix Scenarios** | • Define green, blue, grey hydrogen shares for 2035/2050.<br>• Source $CO_2$ intensity and price
| **Safety & Risk Module** | • Conduct FMEA/HAZOP for hydrogen aircraft.<br>• Model leak detection, fire-suppression, and emerg
| **Operational Impact Analysis** | • Simulate refueling cycles (liquid vs. high-pressure gas).<br>• Estimate ground-support eq
| **Granular Cost Model** | • Adopt ICAO LCOT framework.<br>• Separate CAPEX (aircraft, infrastructure), OPEX (fuel, maintenan
| **Certification Roadmap** | • Map FAA FAR 25, EASA CS-25, ICAO Annex 10 requirements.<br>• Identify safety standards gaps fo
| **Fuel Comparison Matrix** | • Collect cost, emissions, and infrastructure data for SAF, e-fuel, and e-VTOL.<br>• Compare aga
| **Ground-Level Impact Study** | • Use AERMOD or similar to model $NO_x$, PM, and noise at airport perimeters.<br>• Compare with
| **Supply-Chain Assessment** | • Evaluate grid capacity, hydrogen production plants, and material supply (carbon-fiber).<br>•
| **Expanded Stakeholder Engagement** | • Conduct semi-structured interviews with passengers, airport operators, local communit
| **Policy Incentive Modeling** | • Simulate carbon pricing scenarios ($30–$100 $tCO_2^{-1}$).<br>• Model R&D subsidies, tax incentiv
| **Long-Haul Feasibility Study** | • Extend aircraft performance model to trans-Atlantic routes.<br>• Quantify range penaltie
| **Uncertainty & Sensitivity Analysis** | • Define key variables (hydrogen price, fuel cell efficiency, weight penalty, carbon

---

### 4. Revised Deliverable Timeline (High-Level)

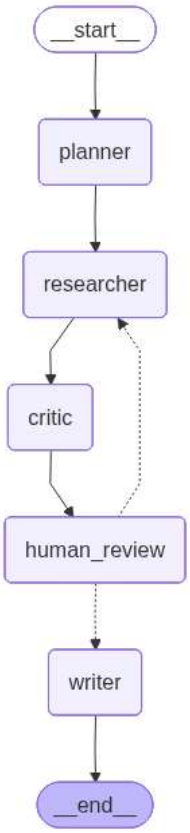
| Month | Milestone |
|-------|-----------|
| 1–2 | Finalize research brief, scope, and production-mix scenarios. |
| 3–4 | Complete safety & risk module; begin operational impact analysis. |
| 5–6 | Deliver granular cost model and certification roadmap. |
| 7–8 | Publish fuel comparison matrix and ground-level impact study. |
| 9–10 | Supply-chain
```

```
print(app.get_graph().draw_mermaid())
```

```
---
config:
  flowchart:
    curve: linear
---
graph TD;
        __start__([<p>__start__</p>]):::first
        planner(planner)
        researcher(researcher)
        critic(critic)
        human_review(human_review)
        writer(writer)
        __end__([<p>__end__</p>]):::last
        __start__ --> planner;
        critic --> human_review;
        human_review -.-> researcher;
        human_review -.-> writer;
        planner --> researcher;
        researcher --> critic;
        writer --> __end__;
        classDef default fill:#f2f0ff,line-height:1.2
        classDef first fill-opacity:0
        classDef last fill:#bfb6fc
```

```
from IPython.display import Image, display

display(Image(app.get_graph().draw_mermaid_png()))
```




```
# https://mermaid.live/edit#pako:eNp1U01vozAQ_SvW7CWRgIaPZsFUuWyue-up6xVyjQ1WwSBjmrZR_vsak0JaaU8zb-bNzBt_nIF1JQcMvu8TxTolZIWJQk
# If flowchart isnt printed above
```